SECTION 7 · CHEAT SHEET

Subagents, hooks and sidecars

Every prompt, command, shortcut and setting from this section, lesson by lesson. Prompts are word for word, as typed on screen.

7.1 Section 7 introduction

Concept lesson: nothing to type, run or set.

7.2 How subagents work

PROMPT

```
Use a research subagent to find out how the cart works on the website: which files hold it, how an item is added, how the total is worked out, and where the checkout reads it. Bring back a short report. Don't change any files.
```

A new conversation in the cafe project. Flash starts a subagent, **Codebase Researcher**, and waits.

SETTING

Side panel › Subagents › Codebase Researcher

Its own conversation; its brief is all it knows. Only the report comes back. No files changed.

7.3 Build in parallel

PROMPT

```
Build three new pages at the same time, each with its own subagent: an FAQ page at /faq that answers common questions about pickup, ordering ahead, opening hours and our two outlets; a diet guide at /diet that lists the menu items by their diet tag from the menu data; and a friendly not-found page for links that don't exist. Each subagent works only on its own page. When all three are done, add FAQ and Diet guide to the header and the mobile menu yourself, then build the site and run the order test.
```

The main agent reads the project first, then starts three specialists in one step.

SETTING

Side panel › Subagents › FAQ Page Specialist

Its brief: only this page, never the header or the mobile menu. Shared files stay with the main agent.

RESULT

Allow run order test suite?

Allow for this conversation: nine checks pass. localhost:3000/brunch shows the not-found page. Check the FAQ's payment and Wi-Fi answers: added unasked.

7.4 Custom specialists

PROMPT

```
Create two custom subagents for this project in .agents/agents/, one Markdown file each, with name, description, model and tools in the frontmatter and its instructions below. A reviewer that checks a change against our house rules in AGENTS.md and reports each rule as passed or failed; it only reads, so its tools are view_file and grep_search. A tester that builds the site and runs the order test, then reports what passed and what failed; its tools are run_command and view_file, and it never edits files. Use flash as the model for both, and give each a clear description so you know when to call it. Everything you need is in this message, so there is nothing to look up, and don't change any other files.
```

.agents/agents/reviewer.md and **tester.md**: the description says when to call each one.

7.4 Custom specialists, in a new conversation

SETTING

Main Agent picker under the prompt box

Main Agent, reviewer, tester: a specialist can lead a whole conversation. We keep Main Agent.

PROMPT

```
On the FAQ page, add a question about how long an order ahead takes: most orders are ready in about 15 minutes. Then have the reviewer check the change against the house rules and the tester build the site and run the order test, and tell me what each one found.
```

Reviewer: five of five. The tester's card waits only briefly; unanswered, its test failed.

PROMPT

```
The order test needs the local server, and the sandbox has no network. Add a line to tester.md telling it to run the order test outside the sandbox and ask me first, then call the tester again and tell me what it found.
```

Fix the specialist's file, not the prompt.

PROMPT

```
Ready now, call the tester again.
```

The card answered at once, **allow for this conversation**: build and nine order tests pass.

7.5 Steering subagents

PROMPT

```
Call the tester to build the site and run the order test, then tell me what it found.
```

It starts the Site and Order Tester. Open it from the side panel.

PROMPT

```
Before you run the order test, also open the FAQ page file and check that the order-ahead answer is there, and add that check to your report.
```

In the tester's own box. It waits under **Queued Messages**; **Send now** hands it over at the next step.

COMMAND

```
agy --model gemini-3.8-flash-high
```

The CLI in the cafe project, on Flash.

COMMAND

```
/agents
```

Lists who can take a job: default, reviewer, tester.

PROMPT

```
Ask the reviewer and the tester to check the FAQ page at the same time: the reviewer against our house rules, the tester with the build and the order test.
```

Both start side by side, with a status line each under the prompt box.

SHORTCUT `Alt+J`

On **tester needs approval**: opens that helper and its waiting command to approve. Esc goes back.

PROMPT

```
@tester also check that the FAQ page still has the answer about pickup
```

A message that starts with a helper's name reaches that helper.

SHORTCUT `Ctrl+K`

Approves the waiting action, here the order test, from the main view.

7.6 Deep reasoning

PROMPT

```
/boost Find out why two orders placed in the same second can get the same order number, and fix it with a test.
```

Docs: Slash commands > Boost deep reasoning. For tricky bugs; it starts at once, with no interview.

RESULT

Typed, never sent

Paid plans. A lead splits one hard problem; helpers fix and test it in clean copies of the project.

7.7 Agent teams

PROMPT

```
/teamwork Build an online ordering system for the cafe, with the menu, a cart, pickup times, an order history page and full test coverage.
```

Docs: Slash commands > Teamwork agent teams. An interview first (what, not how); CLI: Alt+J.

RESULT

Typed, never sent

Paid plans. A coordinator, an orchestrator, explorers and workers, with checks on every milestone.

7.8 The hook lifecycle

Concept lesson: nothing to type, run or set.

7.9 Guard hook

PROMPT

```
Add a guard hook to this project, so the menu and the starting data can never be deleted by mistake. Put it in .agents/hooks.json as a PreToolUse hook on run_command, and have it run a small Python script in .agents/hooks/. The script reads the tool call from its input as JSON. If the command would delete anything in baseline/ or source/, or any menu.csv file, it answers with the decision deny and a short reason saying the menu and the starting data are protected. Anything else gets allow. Then test the script by feeding it two sample tool calls, one harmless and one that deletes baseline/menu.csv. Don't delete anything for real.
```

.agents/hooks/guard.py: every terminal command passes through it first. The sample delete: deny.

PROMPT

```
We don't need the old menu file anymore. Delete baseline/menu.csv.
```

A new conversation, so the hook is loaded. Flash refuses on the house rules.

PROMPT

```
I know, and I still want the file gone. Run the delete command now.
```

Remove-Item: **tool call denied by pre-tool hook**, with our reason. The file stays.

PROMPT

```
Then fix the path instead: since hooks run from the .agents folder, change the command in hooks.json to python hooks/guard.py, remove the second copy, and test once more that deleting baseline/menu.csv is still denied.
```

A hook that can't run fails every command it matches. Its path starts in .agents.

PROMPT

```
Go ahead: first remove the .agents/.agents folder, then run the command that deletes baseline/menu.csv, so we can see the guard hook block it.
```

A new conversation, since hooks load at the start. Denied again; menu.csv intact.

7.10 Quality hook

PROMPT

```
Add a quality hook to this project, next to the guard in .agents/hooks.json. It runs at PostToolUse, after the tools that write or edit files, and it runs a small Python script in .agents/hooks/. The script rebuilds the site with npm --prefix site run build, then runs the order test with python scripts/test_order_flow.py. Give the hook a timeout of 300 seconds. The script writes a short result to .agents/logs/quality.log: whether the build passed and how many order tests passed. If the build or the test fails, it exits with an error and prints the failing lines, so the agent sees them. Remember that a hook command runs from inside the .agents folder. Test the script once by feeding it a sample write_file call.
```

The test drives a real browser, so it asks to run outside the sandbox: **allow for this conversation**.

FILE

.agents/hooks.json

PostToolUse, matcher **write_file|edit_file**, python hooks/quality.py, timeout 300 (the default is 30 s).

PROMPT

```
Add one question to the FAQ page: "Can I bring my own cup?" The answer: yes, any of our hot drinks can go in your own clean cup.
```

A new conversation, so the hook is loaded. The first turn ended empty.

PROMPT

```
Go ahead and add that question and answer to the FAQ page now.
```

About 20 s: the hook builds and tests before the edit counts. quality.log: 9 of 9 passed.

RESULT

A hook blocks the agent

It runs in line on every match, so match only the tools that change files.

7.11 Audit hook

PROMPT

```
Add an audit hook to this project in .agents/hooks.json, next to the guard and the quality hook. It runs at PostToolUse for every tool, and it runs a small Python script in .agents/hooks/ that appends one line per tool call to .agents/logs/audit.log: the step number, the tool name, and a short summary of what the tool was asked to do, like the file path or the command. No dates or times in the log. It must be fast and never fail the tool call, so if anything goes wrong it just writes nothing. Remember that a hook command runs from inside the .agents folder. Test the script once by feeding it a sample view_file call.
```

Matcher *****: every tool. PowerShell broke the sample's quotes, so it wrote nothing, as asked.

PROMPT

```
How many items on the menu are vegan, and which category has the most of them? Just look, don't change anything.
```

A new conversation. Nine vegan items, most in Meals; twelve tool calls in the log.

PROMPT

```
Now read .agents/logs/audit.log and tell me, step by step, what you did to answer that.
```

It rebuilds its own steps from the log, ending with git status and git diff: nothing changed.

FILE

.agents/logs/audit.log

One line per tool call: step number, tool, a short summary. Steps go up by two: each result is a step.

7.12 Sidecars

PROMPT

```
Create a global sidecar called menu-watcher in ~/.gemini/config/sidecars/menu-watcher/. Its sidecar.json runs a small Python script in that folder, with the restart policy on-failure and a short description. The script checks baseline/menu.csv in the cafe project every few seconds, and when the file changes it runs scripts/generate_menu_card.py to rebuild the printed menu card, then prints one line saying it did. Then turn the sidecar on in ~/.gemini/config/config.json, and keep everything else in that file exactly as it is.
```

Default mode asks before each step outside the project: six cards, allowed for this conversation.

FILE

~/.gemini/config/sidecars/menu-watcher/sidecar.json

command **python**, args **watcher.py**, restart_policy **on-failure**; it checks menu.csv every 2 s.

FILE

~/.gemini/config/config.json

sidecars.menu-watcher enabled true; the rest kept. It starts within seconds.

SETTING

Composer › mode menu › Turbo

Request Review, Default, Auto, Turbo: the project's own setting. The cafe runs on Turbo from here.

PROMPT

```
Change the price of Talkie Chai in baseline/menu.csv from 2.25 to 2.50, and change nothing else. Then wait about ten seconds and show me the newest lines of the menu-watcher sidecar's log in ~/.gemini/antigravity/sidecar_data/menu-watcher/logs/.
```

The agent edits only menu.csv; the log shows the menu card rebuilt. No permission card asked.

PROMPT

```
Now put Talkie Chai back to 2.25, wait a few seconds, and show me the log again.
```

A second line: rebuilt again.

SETTING

Customizations › Installed › Automations

menu-watcher with its description and an on/off switch.

7.13 Scheduled sidecars

RESULT

sidecar.json: builtin schedule

Docs: Sidecars. Its args: a cron line (minute, hour, day of month, month, day of week), then the command. **0 * * * *** = every hour, **0 9 * * *** = nine every morning.

RESULT

agentapi

new-conversation starts one, **send-message** adds to one. Needs projectId in config.json.

PROMPT

```
Create a second global sidecar called morning-check in ~/.gemini/config/sidecars/morning-check/. It uses the schedule builtin to run every morning at 9, and what it runs is agentapi new-conversation with this prompt: "Build the cafe site and run the order test, then give me a short report." Set its projectId in ~/.gemini/config/config.json to this cafe project so the conversation opens here, turn it on, and keep everything else in that file exactly as it is.
```

Pasted in a new cafe conversation.

RESULT

Pasted, never sent

Nothing scheduled: every run is a full conversation that uses the plan's usage.